SPECIFICATION V1.0 — DRAFT

BMID
Book Metadata Identifier

A structured, self-describing identifier for LRM entities

Status	Draft, September 2026
Length	25 bytes · 40 Crockford Base32 characters
Payload	RFC 9562 UUID (UUIDv7 recommended)
Integrity	CRC-16/XMODEM over bytes 0–22
Conformance	test-vectors.json

WORKED EXAMPLE

`041ZY000000033SD9A600W93K8NKRKAYDXX8PWNQ`

`01`	`03`	`FF`	`00 00 00 00`	`018F2D4A8C0071239A2B3C4D5E6F7A8B`	`72 B7`
version	Manifestation	issuer	reserved	UUIDv7 payload	CRC

Contents

Summary

Scope

Conformance

Identifier Structure

- Byte Layout
- Canonical Form
- Worked Example
- Worked Example — Wrapped Legacy UUID
- URN Form
- HTTP IRI Form

Invariants Across Versions

Registries

- Version Registry
- Entity Type Registry
- Issuer Registry

Encoding

- Crockford Base32
- Input Normalization
- Sort Order
- Check Bytes
- UUID Payload
- Wrapping Pre-Existing UUIDs

Storage

Validation

- Revocation and Deprecation

Why a Structured Identifier

Carrying BMIDs in Existing Formats

- ONIX for Books 3.x
- MARC 21
- RDF and BIBFRAME
- JSON and other APIs

Related Work and Rejected Alternatives

Governance

Security Considerations

Tools

Changelog

Summary

A BMID is a 25-byte identifier encoded as 40 Crockford Base32 characters. It wraps an RFC 9562 UUID in a fixed envelope that adds the entity type, the minting organization, and a CRC-16. A BMID is not a replacement for UUID — it *contains* one. The envelope makes the identifier routable without a lookup, attributable to its minter, and safe to transcribe.

BMID is designed for a consortium of partner organizations that independently mint identifiers for IFLA Library Reference Model (LRM) entities.

Scope

BMID identifies entities that lack a standard identifier: Works, Expressions, Manifestations, Items, Agents, Series, and Subjects. It does not replace ISBN, ISTC, ISNI, or DOI; those are carried as parallel metadata.

BMIDs are minted by registered **issuers**: metadata aggregators, national bibliographic agencies, large retailers, and similar organizations that maintain entity records on their own authority. Publishers, imprints, and contributors are described *by* BMID-identified records; they do not mint.

Conformance

The key words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, NOT RECOMMENDED, MAY, and OPTIONAL in this document are to be interpreted as described in BCP 14 (RFC 2119, RFC 8174) when, and only when, they appear in all capitals.

The sections *Identifier Structure*, *Invariants Across Versions*, *Registries*, *Encoding*, *Storage*, and *Validation* are **normative**. All other sections are informative.

A conforming implementation accepts exactly the inputs this specification accepts, rejects exactly the inputs it rejects, and produces canonical output as defined here. It MUST pass every vector in `test-vectors.json`.

Identifier Structure

Byte Layout

OFFSET	FIELD	SIZE	DESCRIPTION
0	**Version**	1 byte	BMID format version. See *Version Registry*.
1	**Entity Type**	1 byte	LRM entity class. See *Entity Type Registry*.
2	**Issuer ID**	1 byte	Organization that minted the identifier. See *Issuer Registry*.

OFFSET	FIELD	SIZE	DESCRIPTION
3–6	**Reserved**	4 bytes	MUST be all zero in version 1. Headroom for future versions (e.g. a wider issuer field).
7–22	**UUID Payload**	16 bytes	An RFC 9562 UUID. UUIDv7 RECOMMENDED for new mints.
23–24	**Check Bytes**	2 bytes	CRC-16/XMODEM over bytes 0–22, big-endian.

Total: 25 bytes = 200 bits = 40 Crockford Base32 characters.

The fields carry 21 bytes of information. 168 bits does not divide into 5-bit characters; 25 bytes is the smallest size that does. The surplus is held as reserved bytes rather than padding, and the non-UUID fields lead so that the decoded bytes cluster by version, type, and issuer.

Canonical Form

The canonical form is exactly **40 Crockford Base32 characters**, uppercase, with no separators or surrounding whitespace. Two BMIDs are equal if and only if their canonical forms are byte-equal. Implementations MUST normalize input to canonical form before comparison, hashing, or use as a key.

There is no hyphenated or segmented display form. Field boundaries fall on byte offsets in the decoded buffer, not on character offsets in the string. **Implementations MUST parse fields by decoding the entire string and indexing into the resulting 25-byte array.**

Two visible consequences of the layout:

- The first five characters are the 24 bits of version, entity type, and issuer plus one reserved zero bit. A prefix filter on them selects exactly one (version, entity type, issuer) combination. Version 1 with a core entity type begins `04`.
- Characters 6–11 are `0` in every version 1 BMID (the reserved bytes). For a UUIDv7 payload character 12 is also `0`, since the top four bits of the millisecond timestamp stay zero until 2527.

Worked Example

A Manifestation minted by the testing issuer (`0xFF`):

```
041ZY000000033SD9A600W93K8NKRKAYDXX8PWNQ
```

Decoded bytes (hex):

```
01 03 FF 00 00 00 00 01 8F 2D 4A 8C 00 71 23 9A 2B 3C 4D 5E 6F 7A 8B 72 B7
```

BYTES	VALUE	MEANING
`[0]`	`0x01`	Version = 1
`[1]`	`0x03`	Entity type = Manifestation
`[2]`	`0xFF`	Issuer ID = testing

BYTES	VALUE	MEANING
`[3–6]`	`0x00 0x00 0x00 0x00`	Reserved
`[7–22]`	`01 8F 2D 4A 8C 00 71 23 9A 2B 3C 4D 5E 6F 7A 8B`	UUIDv7 payload
`[23–24]`	`0x72 0xB7`	CRC-16/XMODEM = `0x72B7`

Worked Example — Wrapped Legacy UUID

An Agent whose pre-existing UUIDv4 `f47ac10b-58cc-4372-a567-0e02b2c3d479` is wrapped by the testing issuer:

```
042ZY000000F8YP11DCCRGVJMNKGW0NJRFA7JRJ4
```

```
01 05 FF 00 00 00 00 F4 7A C1 0B 58 CC 43 72 A5 67 0E 02 B2 C3 D4 79 62 44
```

Bytes 7–22 are the original UUID verbatim. Its version nibble (`4` in byte 13) tells any consumer that this payload carries no timestamp.

URN Form

A BMID MAY be expressed as `urn:bmid:<canonical form>`:

```
urn:bmid:041ZY000000033SD9A600W93K8NKRKAYDXX8PWNQ
```

The `urn:` and `bmid:` prefixes are case-insensitive (RFC 8141). The `bmid` namespace is not registered with IANA, so this form is informal. The canonical 40-character form is authoritative.

HTTP IRI Form

For RDF, BIBFRAME, and any context that needs a resolvable identifier, a BMID MAY be expressed as an HTTP IRI whose final path segment is the canonical form. This is the RECOMMENDED form for linked data.

```
https://id.bmid.example/041ZY000000033SD9A600W93K8NKRKAYDXX8PWNQ
```

> [To be decided: the host is a domain owned by the consortium, fixed by this specification once chosen.]

Two IRIs denote the same entity if and only if their path segments are the same canonical form. Resolution behaviour is specified in a companion document. The issuer byte identifies the organization whose resolver is authoritative for an identifier.

Invariants Across Versions

The following hold for every BMID version, present and future. They make it possible to check length, character set, and CRC before the version byte is interpreted.

1. **Length:** 25 bytes; 40 Crockford Base32 characters.
2. **Encoding:** Crockford Base32 with the bit packing defined in *Encoding*.
3. **Version byte:** 1 byte at offset 0.
4. **Check bytes:** CRC-16/XMODEM with the parameters defined in *Check Bytes*, computed over bytes 0–22, stored big-endian at bytes 23–24.

The meaning and layout of bytes 1–22 are defined per version. A future version that needs more room uses the reserved bytes; it MUST NOT change the total length.

Registries

Version, entity type, and issuer codes are allocated from registries maintained in this document. **Allocations are made by pull request against this document**, giving the requested code, the requesting organization, and an organizational contact address. See *Governance*.

Registries are append-only. A code is never deleted or reassigned: identifiers bearing it exist permanently, and reassignment would misattribute them. A retired code is marked *deprecated* and keeps its meaning for existing identifiers.

There is exactly one BMID registry. A code means the same thing in every system that handles BMIDs.

Version Registry

CODE	MEANING
`0x00`	*Reserved — invalid.* MUST be rejected.
`0x01`	Version 1, this specification.
`0x02–0xFF`	*Undefined.* MUST be rejected by version 1 implementations.

Entity Type Registry

CODE	ENTITY	DESCRIPTION
`0x00`	*Reserved*	Invalid; MUST be rejected.
`0x01`	**Work**	An abstract intellectual creation (LRM-E2).
`0x02`	**Expression**	A realization of a Work in a specific form (LRM-E3).
`0x03`	**Manifestation**	A physical or digital embodiment (LRM-E4).
`0x04`	**Item**	A specific exemplar of a Manifestation (LRM-E5).
`0x05`	**Agent**	A person or collective agent (LRM-E6, covering E7 and E8), e.g. a contributor or publisher.
`0x06`	**Series**	A collection of related Works, numbered or unnumbered.
`0x07`	**Subject**	A topic, theme, or classification heading.
`0x08–0x7F`	*Reserved*	Future core types defined by this specification.

CODE	ENTITY	DESCRIPTION
`0x80–0xFE`	*Available*	Registry-assigned extension types.
`0xFF`	*Reserved*	Testing and examples. SHOULD be rejected outside test contexts.

The list is pragmatic rather than a transcription of LRM. Series and Subject get their own codes because they are independently maintained and looked up in practice; Person and Collective Agent share a code because they are typically held in one authority file.

Criterion for a new type. A code is allocated for a class that is *independently resolvable*: it has its own records and its own lookup path. A distinction expressed as an attribute of an existing class (Person versus Collective Agent, print versus digital Manifestation) belongs in metadata, not in the type byte.

The entity type is immutable. A record minted with the wrong type MUST be deprecated in favour of a new BMID of the correct type (see *Revocation and Deprecation*). Issuers SHOULD mint only once an entity's class is settled.

Issuer Registry

CODE	ISSUER
`0x00`	*Reserved — invalid.* MUST be rejected.
`0x01–0xFE`	*Available — registry-assigned.*
`0xFF`	*Reserved for testing and examples.* SHOULD be rejected outside test contexts.

(No partner organizations are allocated in this draft.)

The issuer ID identifies the **minter** at the moment of creation. It is provenance, not current custody. Systems that track custody (after acquisitions, rights transfers, or catalog resale) MUST carry it in a separate mutable metadata field.

Because registries are append-only, the 254 assignable codes are a lifetime budget covering every allocation ever made. Should it approach exhaustion, the reserved bytes allow a future version to widen the field without changing the length.

Encoding

Crockford Base32

The alphabet, in order:

```
0123456789ABCDEFGHJKMNPQRSTVWXYZ
```

It excludes I, L, and O (confusable with 1 and 0) and U. Input is case-insensitive; canonical output is uppercase. It is URL- and filename-safe, and 5 bits per character fits the 200-bit total with no padding.

Bit packing. The 25-byte array is interpreted as a 200-bit unsigned integer, most significant byte first. Each character encodes five bits, most significant group first: the first character carries bits 0–4 of byte 0, the second carries bits 5–7 of byte 0 and bits 0–1 of byte 1, and so on. Decoding is the exact inverse. Libraries that pack least-significant-first, or that pad to an 8-character boundary as RFC 4648 does, are not conformant.

BMID does **not** use Crockford's optional check symbol; the check bytes provide error detection.

Input Normalization

Before validation, implementations MUST, in order:

1. Strip leading and trailing whitespace (any character with the Unicode `White_Space` property).
2. If the string begins with `urn:bmid:` (case-insensitive), remove that prefix.
3. Strip all ASCII hyphens (`-`).
4. Fold to uppercase.
5. Map `I` → `1`, `L` → `1`, and `O` → `0`.

Any remaining character outside the alphabet — including interior whitespace — MUST cause rejection. Normalization applies to input only; the canonical form never contains lowercase letters, hyphens, a URN prefix, or the mapped characters.

Sort Order

The alphabet is in ascending ASCII order, so lexicographic comparison of canonical forms is byte-for-byte equivalent to comparison of the binary forms. Sorting canonical strings groups identifiers by version, entity type, and issuer; within a group, UUIDv7 payloads sort by mint timestamp.

This holds only for the canonical form and only under a **binary (ordinal) collation**. Locale-aware collations MUST NOT be used for columns or indexes holding canonical BMIDs; use, for example, `COLLATE "C"` in PostgreSQL or a `_bin` collation in MySQL. Future revisions MUST NOT introduce an encoding or display form that breaks this equivalence.

Check Bytes

Bytes 23–24 are a **CRC-16/XMODEM** over bytes 0–22, stored big-endian.

PARAMETER	VALUE
Polynomial	`0x1021`
Initial value	`0x0000`
Reflect input	false
Reflect output	false
XOR output	`0x0000`

Test vector: CRC-16/XMODEM of the ASCII string `"123456789"` is `0x31C3`.

The CRC algorithm and parameters are invariant across all versions (see *Invariants Across Versions*), so the CRC can always be checked before the version byte is parsed.

The CRC detects every error confined to 16 consecutive bits of the decoded value. Since each character carries five bits, this covers any error within three adjacent characters, including every single-character error and every adjacent transposition. Other errors are falsely accepted with probability 1 in 65,536. Truncation and padding are rejected by the length check. The CRC detects errors; it does not correct them.

UUID Payload

The 16-byte payload is any valid **RFC 9562 UUID**. Validators enforce the variant bits and a defined version nibble (see *Validation*), not a specific version.

Newly minted identifiers SHOULD use UUIDv7. Its millisecond timestamp gives temporal ordering, time-range queries, and an extractable mint time, and its k-sortability improves index locality. Trade-offs: the mint time is observable to anyone holding the BMID (consider embargoes); entropy is at least 62 random bits rather than 122, which remains safe for uniqueness; burst minting needs a monotonic UUIDv7 implementation.

Any RFC 9562 version is permitted so that pre-existing UUIDs can be wrapped verbatim and recovered from bytes 7–22 without a lookup table. The version nibble travels inside the payload, so every BMID self-describes whether it is time-ordered. Consumers relying on temporal ordering MUST check the version nibble first. Prefix grouping, error detection, routing, and provenance are unaffected by payload version.

Wrapping Pre-Existing UUIDs

Wrapping is deterministic: the same version, entity type, issuer, and UUID always yield the same BMID.

- **Wrap once.** The partner that owns the legacy record mints the wrap. Two partners MUST NOT independently wrap the same UUID; different issuer bytes would yield two BMIDs for one entity. Agree on a single wrapping authority per corpus before migration.
- **The issuer byte identifies the wrapping organization,** not whoever generated the legacy UUID.

What cannot be wrapped. The Nil UUID, the Max UUID, GUIDs using the Microsoft or NCS variant, and arbitrary 128-bit values without RFC 9562 version and variant bits fail structural validation. Such identifiers must be re-keyed with a fresh UUIDv7 and a mapping table maintained by the issuer.

Storage

Three storage strategies are conformant. In all of them, the canonical string is the form that crosses system boundaries.

1. **Binary (25 bytes).** Most compact. Suitable for primary keys and binary protocols.
2. **Canonical string (40 characters).** For APIs, logs, URLs, and display. Columns MUST use a binary collation (see *Sort Order*).
3. **Decomposed.** A BMID is a pure function of (version, entity type, issuer, UUID). A system keyed by UUID keeps that key, adds entity type and issuer as two small columns, and computes the BMID at

its boundary. Reserved and check bytes need not be stored. This is the RECOMMENDED path for wrapped legacy identifiers.

In the decomposed form the issuer column is part of the identity and MUST be stored for every record, never inferred from the system's own identity: a wrapped identifier keeps the issuer that minted the wrap even after the record changes hands.

There is no other display form. Hyphens, spaces, and other separators are not part of the format.

Validation

A conforming implementation MUST validate the following on input, in this order:

1. **Length:** exactly 40 characters after normalization. Otherwise reject.
2. **Character set:** every character MUST be in the Crockford Base32 alphabet. Otherwise reject.
3. **CRC:** decode, recompute CRC-16/XMODEM over bytes 0–22, and compare to bytes 23–24. On mismatch, reject.
4. **Version:** byte 0 MUST be a version this implementation understands. A version 1 implementation accepts only `0x01`; `0x00` and `0x02–0xFF` MUST be rejected.
5. **Entity type:** byte 1 MUST NOT be `0x00`. `0xFF` SHOULD be rejected outside test contexts. Reserved or unallocated codes MAY be accepted, optionally with a warning, so that newly registered types are not rejected by older consumers.
6. **Issuer ID:** byte 2 MUST NOT be `0x00`. `0xFF` SHOULD be rejected outside test contexts. Unallocated codes MAY be accepted, optionally with a warning.
7. **Reserved bytes:** bytes 3–6 MUST all be `0x00`. Otherwise reject. A future version that assigns meaning to these bytes changes the version byte.
8. **UUID version and variant:** the two most significant bits of byte 15 (byte 8 of the payload) MUST be `10`, and the high nibble of byte 13 (byte 6 of the payload) MUST be `0x1` – `0x8`. Otherwise reject. This is a structural check only; it does not require v7.

Steps 1–3 depend only on the invariants, never on the version byte. A production system SHOULD reject the `0xFF` testing codes in steps 5 and 6; a test suite accepts them.

Revocation and Deprecation

BMIDs are **never reused or reassigned.** When an entity is withdrawn, merged, re-typed, or otherwise deprecated, its record is marked deprecated and carries a pointer to the successor BMID, if any. The identifier remains valid as a historical reference and continues to resolve to its deprecation record.

Resolvers MUST distinguish three states: live, deprecated-with-successor, and unknown. Deprecation lives in the metadata layer, not in the identifier.

Why a Structured Identifier

A plain UUID is simpler and sufficient in a system with one entity type, one source of truth, and no human contact with identifiers. In a federated book metadata ecosystem none of those hold: several entity types share a namespace, several partners mint independently, and identifiers appear in search fields, emails, support tickets, and printed materials.

CAPABILITY	BMID	PLAIN UUID
Entity type known without context	Encoded in the identifier	Requires external context or a lookup
Issuer attribution	Encoded in the identifier	Not carried
Error detection	CRC-16; any error within three adjacent characters is caught	None
Temporal ordering	Same as the payload; v7 recommended, version nibble inspectable	Same; v7 if chosen, version nibble inspectable
Global uniqueness	UUID-based	UUID-based
Transcription	Crockford Base32: no ambiguous characters, case-insensitive	Hex with hyphens: 0/O, 1/l, 8/B confusable
Clustering by type and issuer	In both binary and string form	No
Legacy UUID migration	Deterministic wrap, original recoverable	Identity function
String length	40 characters	36 hyphenated, 32 bare
Native database type	None; bytes, string, or decomposed over a UUID column	Ubiquitous
Library support	The decoder in this repository	Every platform
Adoption cost	An encode/decode library; two extra columns or 9 extra bytes per key	None
Re-typing an entity	New identifier plus a deprecation pointer	Not applicable

The last four rows are where a plain UUID wins. BMID is the right choice when identifiers travel between organizations and through human hands often enough that the first eight rows pay for them.

Two design notes follow from the table. First, a BMID pasted into a search field routes to the right service from its first five characters, with no lookup and no network call. Second, type and issuer make poor *sharding* keys — there are few of them and they are unevenly loaded — so shard on the UUID payload and use the prefix for pruning.

Carrying BMIDs in Existing Formats

BMID requires no new message formats. In every case below the value carried is the canonical 40-character string.

ONIX for Books 3.x

ENTITY	WHERE	HOW
Manifestation	`<ProductIdentifier>`	`<ProductIDType>` 01 (proprietary), `<IDTypeName>` `BMID`, `<IDValue>` the BMID
Work	`<RelatedWork>` with `<WorkRelationCode>` 01 (manifestation of), `<WorkIdentifier>`	`<WorkIDType>` 01 (proprietary), `<IDTypeName>` `BMID`
Series	`<Collection>`, `<CollectionIdentifier>`	`<CollectionIDType>` 01 (proprietary), `<IDTypeName>` `BMID`
Agent (contributor)	`<Contributor>`, `<NameIdentifier>`	`<NameIDType>` 01 (proprietary), `<IDTypeName>` `BMID`
Agent (publisher, imprint)	`<Publisher>` or `<Imprint>`, `<NameIdentifier>`	`<NameIDType>` 01 (proprietary), `<IDTypeName>` `BMID`
Subject	`<Subject>`	`<SubjectSchemeIdentifier>` 24 (proprietary), `<SubjectSchemeName>` `BMID`, `<SubjectCode>` the BMID
Expression, Item	—	No natural ONIX home; ONIX describes products, not expressions or copies.

The `<IDTypeName>` and `<SubjectSchemeName>` value is the fixed token `BMID`, per EDItEUR's guidance for proprietary identifiers (*Identifiers in ONIX*, 2025).

MARC 21

Field `024` with first indicator `7`, `$a` the BMID, and `$2` a source code for BMID, in bibliographic and authority records alike. *[To be decided: register the source code with the Library of Congress Standard Identifier Source Codes list.]*

RDF and BIBFRAME

Use the HTTP IRI form as the resource IRI, or attach it via `bf:identifiedBy` with an identifier resource whose value is the canonical string. The URN form MAY be used where an IRI is required but resolution is not.

JSON and other APIs

Carry the canonical string. Never carry the binary form in JSON.

Related Work and Rejected Alternatives

- **UUIDv7 with a type column.** Works inside one system; type and source are lost when the identifier leaves it, and there is no error detection. BMID is this design plus an envelope that keeps the context attached.
- **ULID.** Same encoding family, no type, issuer, or check bytes, and not an RFC 9562 UUID.
- **TypeID.** The closest relative: a readable type prefix, an underscore, and a UUIDv7 in 26 Crockford characters. Variable-length, no issuer, no check bytes, no fixed-width binary form. BMID trades the readable prefix for fixed width, provenance, and error detection.
- **ARK.** Its name assigning authority number plays the role of the issuer byte. ARK is a resolvable URL scheme over variable-length names, not a fixed binary key; a BMID can serve as the name part of an ARK.
- **DOI and Handle.** Resolution infrastructure with registration agencies and per-identifier fees, suited to published objects rather than to internal entity keys minted at scale.
- **CRC-32.** Two extra bytes would consume half the reserved headroom for a false-acceptance rate already far below what transcription errors demand.
- **A check character.** A Damm or Crockford mod-37 symbol detects single errors and adjacent transpositions and nothing else; the CRC detects strictly more for one additional byte.
- **An environment or partition byte.** Tried and removed. It carried a constant for single-environment partners and baked unstable operational structure into permanent identifiers. Environment belongs in the system that stores the record, not in its key.

Governance

> [Items marked "to be decided" need an answer before v1.0 is final.]

- **Owner.** *[To be decided: the consortium body or lead organization that owns this specification and its registries.]*
- **Registry maintenance.** This repository is the registry. Allocation requests are pull requests against this document. *[To be decided: the maintainers who review them, and the review period.]*
- **Issuer allocation criteria.** The requesting organization maintains entity records on its own authority, provides an organizational contact, and agrees to the wrap-once rule and to never reusing an identifier.
- **Entity type allocation criteria.** As stated in *Entity Type Registry*.
- **Change policy.** Registries are append-only. A change to any normative section that alters which inputs are accepted, or what they mean, is a new version and takes a new version byte. Editorial changes and additions to informative sections do not.
- **License.** *[To be decided. CC BY 4.0 for this document and Apache-2.0 or MIT for reference code are the conventional choices.]*

Security Considerations

The check bytes detect errors, not tampering. Where cryptographic integrity is required, layer HMAC or signatures over the BMID rather than extending it.

The issuer ID is informational and MUST NOT be used as an access control mechanism. It also discloses the original minter, which may be commercially sensitive; partners exposing BMIDs externally should consider whether that is acceptable.

A UUIDv7 payload discloses the mint time to the millisecond. Wrapped legacy payloads disclose whatever their original version encodes; a v1 UUID embeds a timestamp and historically a MAC address.

Tools

- `test-vectors.json` — the conformance suite: valid identifiers with decoded fields, normalization cases, and invalid inputs keyed to the validation step that rejects them.
- `tools/bmid-decoder.html` — a browser bookmarklet that normalizes, validates, and decodes a BMID.

Changelog

- **2026-09-30** — Editorial. Condensed the whole document; merged *The Problem with Plain UUIDs* and *Key Advantages* into *Why a Structured Identifier*; removed the glossary. Explained why the identifier is 25 bytes. No normative change.
- **2026-09-24** — Renamed the vendor field to *Issuer ID*. Added *Conformance*, *Invariants Across Versions*, *Version Registry*, *HTTP IRI Form*, *Carrying BMIDs in Existing Formats*, *Related Work and Rejected Alternatives*, *Governance*, and *Tools*. Specified Base32 bit packing, the URN prefix in normalization, and Unicode whitespace. Made non-zero reserved bytes and unknown versions a hard reject. Added the decomposed storage strategy, the binary-collation requirement, the entity type criterion and immutability rule, and what cannot be wrapped. Stated the CRC guarantee precisely. Rebalanced the comparison table. Added `test-vectors.json`.
- **2026-07-30** — Removed the environment/partition byte; bytes 3–6 are reserved and zero. Allowed any RFC 9562 UUID as payload to support wrapping. Made normalization mandatory, registries append-only, and documented the sort-order guarantee.
- **2026-04-22** — Initial draft.